AGRO HUB · BACKEND API

API Reference

A complete, endpoint-by-endpoint reference for the Agro Hub REST API — authentication, request/response schemas, and error contracts for every route the mobile client consumes.

Version 1 (`/api/v1`) · Generated 2026-09-19 · codecell.site

Agro Hub API Reference

Base URL: `https://codecell.site/api/v1`

Transport: HTTPS. Request/response bodies are `application/json` unless an endpoint accepts file uploads, in which case it takes `multipart/form-data` (noted per endpoint).

Authentication: Bearer token issued by Laravel Sanctum. Obtain one from `POST /auth/register` or `POST /auth/login`, then attach it to every subsequent request:

```
Authorization: Bearer 1|xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
Accept: application/json
```

Tokens do not expire server-side and are not rotated; a token is revoked only by `POST /auth/logout` (which revokes the token used on that request) or by deleting the underlying `personal_access_tokens` row. A user may hold multiple valid tokens simultaneously (one per signed-in device) — logging in on a new device does not invalidate other sessions.

Versioning: All routes are namespaced under `/v1` in the path. There is no header-based content negotiation for version.

Conventions

Pagination

Every list endpoint marked **Paginated** returns Laravel's standard length-aware paginator envelope, produced by wrapping an Eloquent collection in a `Resource::collection()`:

```
{
  "data": [ /* array of resource objects */ ],
  "links": {
    "first": "https://codecell.site/api/v1/listings?page=1",
    "last": "https://codecell.site/api/v1/listings?page=4",
    "prev": null,
    "next": "https://codecell.site/api/v1/listings?page=2"
  },
  "meta": {
    "current_page": 1,
    "from": 1,
    "last_page": 4,
    "links": [ /* per-page link objects */ ],
    "path": "https://codecell.site/api/v1/listings",
    "per_page": 20,
    "to": 20,
    "total": 73
  }
}
```

Request a specific page with the query parameter `?page=N`. Page size is a fixed server-side constant per endpoint (listed in each section) — it is **not** client-configurable except where an endpoint explicitly documents a `per_page` parameter.

Timestamps

All `*_at` fields are ISO 8601 strings, e.g. `"2026-09-19T08:15:30+00:00"`.

Idempotency

This API has no `Idempotency-Key` header mechanism. A handful of mutating endpoints are naturally idempotent by design (documented per endpoint) — e.g. saving an already-saved listing, or re-marking an already-read notification as read.

Error shapes

STATUS	MEANING	BODY
`400`	Malformed request	`{"message": "..."}`
`401`	Missing, invalid, or revoked bearer token	`{"message": "Unauthenticated."}`
`403`	Authenticated but forbidden	two distinct shapes — see below
`404`	Resource does not exist, or is not visible to the caller	`{"message": "No query results for model [App\\Models\\Listing] 42"}`
`422`	Validation failure	`{"message": "The given data was invalid.", "errors": {"field": ["Human-readable reason."]}}`
`500`	Unhandled server exception	`{"message": "Server Error"}` in production (`APP_DEBUG=false`)

401 is emitted globally for any `api/*` route by a custom `AuthenticationException` renderer (`bootstrap/app.php`) — this guarantees a clean JSON body instead of Laravel's default HTML login-redirect, regardless of the request's `Accept` header.

403 has two distinct causes and bodies, both possible on almost every authenticated endpoint:

- **Suspended account** — `App\Http\Middleware\EnsureUserIsNotSuspended` (alias `not_suspended`) blocks the request before the controller runs:

  ```
  { "message": "This account has been suspended." }
  ```

  This middleware wraps every authenticated route **except** `GET /auth/me` , `POST /auth/logout` , `DELETE /auth/me` , `POST /device-tokens` , and `DELETE /device-tokens` — a suspended account can still check its own status, sign out, delete its account, and unregister its device.

- **Authorization policy denial** — Laravel's default `AuthorizationException` message:

  ```
  { "message": "This action is unauthorized." }
  ```

`422` validation errors follow Laravel's default `ValidationException` shape; the `errors` object maps each invalid field to an array of one or more human-readable messages.

Admin bypass

Every ownership-gated Policy in this API (`ListingPolicy` , `GroupPolicy` , `PostPolicy` , `CommentPolicy` , `EventPolicy` , `SwapOfferPolicy` , `AppNotificationPolicy`) grants full access to any account with `is_admin = true` via a `before()` hook, letting staff act on any user's content from the same endpoints the mobile app uses. `AiConversationPolicy` **is the sole exception** — it has no admin bypass; an AI conversation is only ever visible to the user who owns it.

Geolocation & fuzzy locations

Every location-bearing resource (`User` , `Listing`) stores an exact coordinate server-side (`base_location` , **never** returned by any endpoint) and a public-facing `location: {latitude, longitude}` that has been randomly offset by 500–1000 meters (`fuzzy_location`). This is a deliberate privacy measure — the API never exposes a user's or listing's exact handover point. A user can hide even the fuzzy location from other users entirely via their `show_handover_radius` profile flag; when `false` , `location` renders as `null` to anyone but the owner.

Authentication

`POST /auth/register`

Creates a new account and returns a bearer token. No authentication required.

Headers

HEADER	REQUIRED	VALUE
`Content-Type`	Yes	`application/json`
`Accept`	Yes	`application/json`

Request body

FIELD	TYPE	REQUIRED	NOTES
`name`	string	Yes	Max 255 chars.
`email`	string	Yes	Valid email, max 255 chars, must be unique.
`password`	string	Yes	Must satisfy Laravel's default `Password` policy; must be accompanied by `password_confirmation` with an identical value (`confirmed` rule).
`password_confirmation`	string	Yes	Must match `password`.
`latitude`	number	Yes	Between -90 and 90. Mandatory — every account requires a location to participate in this hyper-local app.
`longitude`	number	Yes	Between -180 and 180.
`location_label`	string	No	Free-text label, e.g. `"Dhanmondi, Dhaka"`. Max 255 chars.
`experience_level`	string	No	One of `beginner`, `intermediate`, `experienced`, `expert`.
`growing_setup`	string	No	One of `balcony`, `indoor`, `backyard`, `rooftop`, `community_garden`, `greenhouse`.
`device_name`	string	No	Label for the issued token (defaults to `"mobile"`). Max 255 chars.

```
{
  "name": "Amina Rahman",
  "email": "amina@example.com",
  "password": "Str0ng!Passw0rd",
  "password_confirmation": "Str0ng!Passw0rd",
  "latitude": 23.7461,
  "longitude": 90.3742,
  "location_label": "Dhanmondi, Dhaka",
  "experience_level": "intermediate",
  "growing_setup": "rooftop",
  "device_name": "Amina's Pixel 8"
}
```

Response `201 Created`

```
{
  "data": {
    "id": 154,
    "name": "Amina Rahman",
    "email": "amina@example.com",
    "phone": null,
    "show_handover_radius": true,
    "is_suspended": false,
    "suspension_reason": null,
    "is_verified": false,
    "username": null,
    "bio": null,
    "avatar_url": null,
    "cover_photo_url": null,
    "location_label": "Dhanmondi, Dhaka",
    "growing_interests": [],
    "experience_level": "intermediate",
    "growing_setup": "rooftop",
    "rating_avg": 0.0,
    "location": { "latitude": 23.7469, "longitude": 90.3751 },
    "created_at": "2026-09-19T08:15:30+00:00"
  },
  "token": "1|xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
  "token_type": "Bearer"
}
```


`email`, `phone`, `show_handover_radius`, `is_suspended`, and `suspension_reason` are included here because this response represents the caller's own account (forced via `UserResource::asSelf()`) — they are omitted when any other user's `UserResource` is returned elsewhere in this API.

Errors

`422` — e.g. duplicate email:


```
{
  "message": "The given data was invalid.",
  "errors": { "email": ["The email has already been taken."] }
}
```

`POST /auth/login`

Issues a new bearer token for an existing account. Does **not** revoke tokens already issued to other devices — a user may be signed in on several devices concurrently. No authentication required.

Request body

FIELD	TYPE	REQUIRED	NOTES
`email`	string	Yes	Valid email.
`password`	string	Yes	
`device_name`	string	No	Label for the issued token (defaults to `"mobile"`). Max 255 chars.

```
{ "email": "amina@example.com", "password": "Str0ng!Passw0rd", "device_name": "Amina's Pixel 8" }
```

Response `200 OK` — identical shape to register's `201`, minus `Location:` semantics (no `Location` header; status is simply `200`).

Errors

`422` — wrong email or password (deliberately generic, doesn't reveal which):

```
{
  "message": "The given data was invalid.",
  "errors": { "email": ["The provided credentials are incorrect."] }
}
```

`403` — correct credentials, but the account is suspended (checked **before** the token is issued — a suspended account's login attempt never receives a working token):

```
{ "message": "This account has been suspended: Repeated marketplace scam reports." }
```

If no `suspension_reason` was set by the admin, the body is the generic `{"message": "This account has been suspended."}`.

`GET /auth/me`

Returns the authenticated account, including two owner-only counts. Used by the mobile app both to restore a session (validate a stored token) and to detect a suspension applied after the token was issued. **Not** blocked by `not_suspended` — a suspended account must still be able to see its own status.

Headers: `Authorization: Bearer <token>` (required).

Response `200 OK`

```
{
  "data": {
    "id": 154,
    "name": "Amina Rahman",
    "email": "amina@example.com",
    "phone": null,
    "show_handover_radius": true,
    "is_suspended": false,
    "suspension_reason": null,
    "is_verified": false,
    "username": "amina_grows",
    "bio": "Rooftop gardener, Dhaka.",
    "avatar_url": "https://codecell.site/storage/avatars/8f3c1e2a-....jpg",
    "cover_photo_url": null,
    "location_label": "Dhanmondi, Dhaka",
    "growing_interests": ["tomatoes", "basil"],
    "experience_level": "intermediate",
    "growing_setup": "rooftop",
    "rating_avg": 4.8,
    "location": { "latitude": 23.7469, "longitude": 90.3751 },
    "posts_count": 12,
    "active_listings_count": 3,
    "created_at": "2026-01-04T10:02:00+00:00"
  }
}
```


`posts_count` and `active_listings_count` are present **only** on this endpoint and `PATCH /auth/me` (both explicitly load these counts) — they are absent from `UserResource` payloads returned elsewhere (e.g. embedded in a listing's `owner`).

Errors: `401` if the token is missing/invalid.

`PATCH /auth/me`

Partial update of the caller's own profile. Every field is optional (`sometimes`) — send only what changed. Accepts `multipart/form-data` when uploading `avatar` / `cover_photo`, or plain JSON otherwise.

Request body

FIELD	TYPE	REQUIRED	NOTES
`name`	string	No	Max 255.
`username`	string | null	No	Max 30, pattern `^[a-zA-Z0-9_]+$`, must be unique (excluding the caller's own current row).
`bio`	string | null	No	Max 300.
`phone`	string | null	No	Max 30.
`location_label`	string | null	No	Max 255.
`growing_interests`	string[] | null	No	Max 10 items, each max 40 chars.
`show_handover_radius`	boolean	No	Accepts any of `true` / `false` / `"true"` / `"false"` / `1` / `0` / `"1"` / `"0"` — normalized server-side since multipart form data has no native boolean type.
`experience_level`	string | null	No	`beginner` | `intermediate` | `experienced` | `expert`.
`growing_setup`	string | null	No	`balcony` | `indoor` | `backyard` | `rooftop` | `community_garden` | `greenhouse`.
`avatar`	file	No	JPEG/PNG/WebP, max 8192 KB.
`cover_photo`	file	No	JPEG/PNG/WebP, max 8192 KB.

Response `200 OK` — same shape as `GET /auth/me`, reflecting the new values.

Errors

`422` — e.g. taken username:

```
{ "message": "The given data was invalid.", "errors": { "username": ["The username has already been ta
```

`POST /auth/logout`

Revokes the Sanctum token used to authenticate this specific request (signs out this device only — other devices' tokens are untouched). Not blocked by `not_suspended`.

Response `200 OK`

```
{ "message": "Logged out." }
```

`DELETE /auth/me`

Permanently deletes the caller's own account and personal data (profile, listings, posts, comments, events, chat messages, AI conversations, uploaded photos, device tokens). Requires the current password so a stolen token alone cannot wipe an account. Cancels any live Stripe subscription first. **Not** blocked by `not_suspended` — a suspended account can still delete itself.

Groups the caller created that still have other members are handed to another member (an existing admin, else the longest-standing member) rather than deleted; a group with no other members is deleted with its creator.

Request body

FIELD	TYPE	REQUIRED
`password`	string	Yes — the account's current password.

Response `200 OK`

```
{ "message": "Your account has been deleted." }
```

Errors

`422` — missing or wrong password:

```
{ "message": "The password is incorrect.", "errors": { "password": ["The password is incorrect."] } }
```

`503` — the subscription could not be cancelled (or another failure occurred); the account is left untouched so the user isn't deleted while still being billed:

```
{ "message": "Your account could not be deleted right now. Please try again shortly." }
```

Trust & Safety

User-generated-content moderation tooling (reporting and blocking). Reports land in the admin panel's **Reports** queue; a reported listing is also flagged in the listing-moderation queue.

`POST /reports`

Reports a piece of content or a user. Idempotent per `(reporter, target)`: re-reporting the same item returns `200` without creating a second report.

Request body

FIELD	TYPE	REQUIRED	NOTES
`target_type`	string	Yes	`post` | `comment` | `listing` | `event` | `group` | `user` | `group_message` | `message` (a swap-offer chat message).
`target_id`	integer	Yes	Id of the reported row.
`reason`	string	Yes	`spam` | `inappropriate` | `harassment` | `misleading` | `illegal` | `other`.
`details`	string | null	No	Max 500.

```
{ "target_type": "post", "target_id": 3021, "reason": "spam", "details": "Looks like an ad." }
```

Response `201 Created` (new report) or `200 OK` (already reported)

```
{ "message": "Thanks — our team will review this report." }
```

Errors

`422` — the target doesn't exist, is the caller's own content (or the caller themselves), or a field is invalid:

```
{ "message": "You cannot report your own content.", "errors": { "target_id": ["You cannot report your
```

`POST /users/{user}/block` · `DELETE /users/{user}/block`

Blocks / unblocks a user. Idempotent. Blocking is **mutual in effect**: each side stops seeing the other's posts, comments, listings, events, group messages, and profile (`GET /users/{user}` and `/users/{user}/listings` return `404`), swap-offer threads between them drop out of both inboxes, neither can make a swap offer to or message the other (`403`), and neither triggers notifications for the other.

Response `200 OK`

```
{ "message": "User blocked." }
```

or

```
{ "message": "User unblocked." }
```

Errors: `422` when blocking yourself — `{"errors": {"user": ["You cannot block yourself."]}}` .

`GET /blocks`

The accounts the caller has blocked, most recently blocked first. **Paginated**, 50/page.

Response `200 OK` — paginated `UserResource` collection.

Device Tokens

Registers a device's Firebase Cloud Messaging token so the backend can deliver push notifications for likes, comments, replies, and chat messages. Neither route is blocked by `not_suspended` — a suspended account must still be able to unregister on logout.

`POST /device-tokens`

Upserts **by** `token` **alone**, not `(user_id, token)` — if the same physical device token was previously registered under a different account (a shared device, or a re-login as someone else), this reassigns ownership so the old account stops receiving that device's pushes.

Request body

FIELD	TYPE	REQUIRED	NOTES
`token`	string	Yes	Max 255. The FCM registration token.
`platform`	string	No	`android` or `ios`. Defaults to `android`.
`device_id`	string	No	Max 255.
`app_version`	string	No	Max 32.

Response `200 OK`

```
{ "message": "Device token registered." }
```

`DELETE /device-tokens`

Deletes the token row matching **both** the given `token` and the caller's own `user_id` — one account cannot delete another's row even if it somehow obtained the token value.

Request body

FIELD	TYPE	REQUIRED
`token`	string	Yes

Response `200 OK`

```
{ "message": "Device token removed." }
```

Config

`GET /config/stripe-key`

Public, unauthenticated. Returns the live Stripe publishable key and the current Agri-AI Premium display prices, so the mobile app never hardcodes either at build time — an admin rotating the key or changing a price from the web panel takes effect immediately with no app update required.

Response `200 OK`

```
{
  "publishable_key": "pk_live_51N...",
  "monthly_price_usd": 9.99,
  "annual_price_usd": 71.88
}
```

Users

`GET /users/{user}`

A grower's public profile, e.g. reached from "View Seller Profile" on a listing card. Exposes only what `UserResource` already treats as public — never email, phone, or suspension state.

Path parameters: `user` (integer, required) — the target user's id.

Response `200 OK` — a `UserResource` object (see the shape under `GET /auth/me`, minus `email` / `phone` / `show_handover_radius` / `is_suspended` / `suspension_reason` / `posts_count` / `active_listings_count`, all of which are self-only or count-only fields not loaded on this endpoint).

Errors: `404` if no such user.

`GET /users/{user}/listings`

That user's **active** marketplace listings only (`status: active`), newest first. **Paginated**, 20/page.

Response `200 OK` — paginated envelope of `ListingResource` objects (see Marketplace section below). `owner` is present on each item (relation is eager-loaded here, unlike `GET /listings`).

Marketplace

`GET /listing-categories`

The full admin-managed category list (used to populate the "create listing" category picker). Not paginated — always a short, admin-curated reference list.

Response `200 OK`

```
{
  "data": [
    { "id": 1, "name": "Seeds & Scions" },
    { "id": 2, "name": "Live Plants & Saplings" },
    { "id": 3, "name": "Organic Fertilizers" },
    { "id": 4, "name": "Farm Tools & Equipment" },
    { "id": 5, "name": "Harvest & Produce" },
    { "id": 6, "name": "Compost & Soils" }
  ]
}
```

`listing_type` on a listing is a free-form string matched against this table's `name` column (`Rule::exists`), not a fixed enum — an admin can add/rename categories at any time from the web panel.

`GET /listings/nearby`

Geospatial discovery search — the marketplace's primary browse endpoint. **Paginated**, default 20/page (client-adjustable via `per_page`).

Query parameters

PARAM	TYPE	REQUIRED	NOTES
`latitude`	number	Yes	Between -90 and 90.
`longitude`	number	Yes	Between -180 and 180.
`radius_km`	number	No	Between 1 and 50. Defaults to 10.
`category`	string	No	Must match a `listing_categories.name` row.
`transaction_type`	string	No	`swap` | `free` | `sell` | `swap_or_cash` | `wanted`.
`per_page`	integer	No	Between 1 and 50. Defaults to 20.
`sort`	string	No	`nearest` (default) or `newest`.
`q`	string	No	Max 255. Free-text search against title/variety/looking_for/description.

Response `200 OK` — paginated envelope of `ListingResource` objects. Each item includes `distance_km` (only on this endpoint, and only when the listing's location is visible to the caller — otherwise the key is entirely omitted, not `null`).

`GET /listings/saved`

The caller's bookmarked listings, most-recently-saved first. **Paginated**, 20/page. Excludes logically-deleted listings.

Response `200 OK` — paginated `ListingResource` collection; every item has `is_saved: true`.

`GET /listings`

The caller's **own** listings — including reserved/completed/deleted ones (this is a management view, not the discovery feed). **Paginated**, 20/page, newest first.

Response `200 OK` — paginated `ListingResource` collection. `owner` **key is absent** on this endpoint (the `user` relation isn't eager-loaded, since it's always the caller).

`POST /listings`

Creates a new marketplace listing. `multipart/form-data` when uploading photos.

Request body

FIELD	TYPE	REQUIRED	NOTES
`title`	string	Yes	Max 255. Checked against the banned/invasive-species list (see below).
`variety`	string | null	No	Max 255. Also checked against the banned-species list.
`description`	string | null	No	Max 2000.
`condition`	string | null	No	`new_fresh` | `established_potted` | `lightly_used` | `organic_certified`.
`listing_type`	string	Yes	Must match a `listing_categories.name` row.
`transaction_type`	string	Yes	`swap` | `free` | `sell` | `swap_or_cash` | `wanted`.
`price`	number	Conditional	**Required when** `transaction_type` **is** `sell` **or** `swap_or_cash`; otherwise optional. Min 0, max 99999.99.
`looking_for`	string | null	No	Max 1000.
`handover_note`	string | null	No	Max 500.
`latitude`	number	No	Between -90 and 90. Required together with `longitude` if either is sent. Defaults to the poster's own account location if omitted.
`longitude`	number	No	Between -180 and 180.
`photos`	file[]	No	Max 6 files, each JPEG/PNG/WebP, max 8192 KB.

```
{
  "title": "Cherry Tomato Seedlings",
  "variety": "Sweet Million",
  "description": "6 healthy seedlings, ~4 weeks old, grown organically.",
  "condition": "established_potted",
  "listing_type": "Live Plants & Saplings",
  "transaction_type": "swap_or_cash",
  "price": 5.00,
  "looking_for": "Basil or mint cuttings",
  "handover_note": "Available evenings after 6pm, Dhanmondi 27"
}
```

Response `201 Created` — a `ListingResource` object:

```
{
  "data": {
    "id": 512,
    "title": "Cherry Tomato Seedlings",
    "variety": "Sweet Million",
    "description": "6 healthy seedlings, ~4 weeks old, grown organically.",
    "condition": "established_potted",
    "condition_label": "Established / Potted",
    "listing_type": "Live Plants & Saplings",
    "transaction_type": "swap_or_cash",
    "price": 5.0,
    "status": "active",
    "looking_for": "Basil or mint cuttings",
    "handover_note": "Available evenings after 6pm, Dhanmondi 27",
    "location": { "latitude": 23.7469, "longitude": 90.3751 },
    "views_count": 0,
    "photos": [],
    "is_saved": false,
    "owner": { "...": "UserResource" },
    "created_at": "2026-09-19T08:20:00+00:00",
    "updated_at": "2026-09-19T08:20:00+00:00"
  }
}
```


Errors

`422` — banned species reference:


```
{
  "message": "The given data was invalid.",
  "errors": { "title": ["The title cannot reference \"Japanese Knotweed\", which is a banned or invasi
}
```


`422` — no location available at all (neither sent in the request nor on file for the account — practically unreachable since registration requires one, but enforced defensively):


```
{ "message": "The given data was invalid.", "errors": { "latitude": ["A location is required; none was
```

`GET /listings/{listing}`

Path parameters: `listing` (integer, required).

Increments `views_count` unless the viewer is the listing's own owner (self-views never count).

Response `200 OK` — a `ListingResource` object, with `is_saved` computed live for the caller.

Errors: `404` if the listing is `deleted` and the caller isn't its owner (owners may still view their own deleted listings).

`PATCH /listings/{listing}`

Partial update — every field is `sometimes`. Only the listing's owner (or an admin) may update it. New photos are **appended** to existing ones (not replaced), continuing the `sort_order` sequence.

Request body — same fields as `POST /listings` (all optional), plus:

FIELD	TYPE	REQUIRED	NOTES
`status`	string	No	`active` | `reserved` | `completed` | `deleted`. Settable only via update, not create. No transition restrictions are enforced — any value is accepted from an authorized caller.

Response `200 OK` — updated `ListingResource`.

Errors: `403` if the caller doesn't own the listing (and isn't an admin).

`DELETE /listings/{listing}`

Logical delete only — sets `status: deleted`; the row is retained so existing swap-offer/message history referencing it stays intact.

Response `200 OK`

```
{ "message": "Listing deleted." }
```

Errors: `403` if the caller doesn't own the listing.

`POST /listings/{listing}/save` · `DELETE /listings/{listing}/save`

Bookmarks / un-bookmarks a listing for the caller. Both are idempotent (saving an already-saved listing, or unsaving an already-unsaved one, is a no-op — no error). No ownership restriction; any authenticated user may save any listing, including their own.

Response `200 OK` — the `ListingResource`, with `is_saved` reflecting the new state.

`POST /listings/{listing}/swap-offers`

Opens (or reuses) a negotiation thread on a listing.

Request body

FIELD	TYPE	REQUIRED
`offer_details`	object | null	No
`offer_details.offered_listing_id`	integer	No — must reference an existing listing.
`offer_details.note`	string	No — max 1000.

```
{ "offer_details": { "note": "Would you swap for a basil cutting instead?" } }
```

Response `201 Created` on a genuinely new offer; `200 OK` (same `SwapOfferResource` body, no new row created) if the caller already has a `pending` / `countered` / `accepted` offer on this listing — reopening that existing thread rather than spawning a duplicate.

```
{
  "data": {
    "id": 88,
    "status": "pending",
    "status_label": "Pending",
    "offer_details": { "note": "Would you swap for a basil cutting instead?" },
    "listing": { "...": "ListingResource" },
    "requester": { "...": "UserResource" },
    "created_at": "2026-09-19T08:25:00+00:00",
    "updated_at": "2026-09-19T08:25:00+00:00"
  }
}
```

Errors

`422` — offering on your own listing:

```
{ "message": "The given data was invalid.", "errors": { "listing_id": ["You cannot make an offer on yo
```

`422` — listing not currently active:

```
{ "errors": { "listing_id": ["This listing is not currently accepting offers."] } }
```

`GET /swap-offers`

Every swap offer where the caller is either the requester or the listing's owner. **Paginated**, 20/page, newest first.

Response `200 OK` — paginated `SwapOfferResource` collection. Each item includes `unread_count` and `last_message` (a compact preview object: `{body, has_image, is_system_event, sender_id, created_at}`, or `null`) — both present **only** on this endpoint.

`GET /swap-offers/{swapOffer}`

Response `200 OK` — a `SwapOfferResource`, including `unread_count` but **not** `last_message` (only populated on the index).

Errors: `403` if the caller is neither the requester nor the listing's owner.

`PATCH /swap-offers/{swapOffer}/status`

Advances the offer through its state machine. Every transition posts an auto-generated system message into the thread.

Request body

FIELD	TYPE	REQUIRED
`status`	string	Yes — `pending` | `countered` | `accepted` | `declined` | `completed` | `canceled`.
`offer_details`	object | null	No — only applied when `status` is `countered`.

Valid transitions (enforced by policy, not by the request validator):

FROM	LISTING OWNER MAY SET	REQUESTER MAY SET
`pending`	`accepted`, `declined`, `countered`	`canceled`
`countered`	`canceled`	`accepted`, `declined`
`accepted`	`completed`, `canceled`	`completed`, `canceled`
`declined` / `completed` / `canceled`	— (terminal)	— (terminal)

Response `200 OK` — updated `SwapOfferResource`.

Errors: `403` for any transition not in the table above, e.g. a requester trying to `accept` their own still-`pending` offer.

`GET /swap-offers/{swapOffer}/messages`

The offer's chat thread. **Paginated**, 50/page — returns the **most recent** page in chronological order (mimics opening a chat app at the bottom of the conversation). Opening this endpoint marks every unread message from the other party (and every system event) as read.

Response `200 OK` — paginated `MessageResource` collection.


```
{
  "data": [
    {
      "id": 301,
      "swap_offer_id": 88,
      "message_type": "system_event",
      "body": "Amina Rahman made an offer on \"Cherry Tomato Seedlings\".",
      "image_url": null,
      "sender": null,
      "created_at": "2026-09-19T08:25:00+00:00"
    },
    {
      "id": 302,
      "swap_offer_id": 88,
      "message_type": "text",
      "body": "Sure, that works!",
      "image_url": null,
      "sender": { "...": "UserResource" },
      "created_at": "2026-09-19T08:31:00+00:00"
    }
  ]
}
```


Errors: `403` if the caller is neither party to the offer.

`POST /swap-offers/{swapOffer}/messages`

Sends a chat message on the thread — works even on a terminal (`declined` / `completed` / `canceled`) offer. `multipart/form-data` for image attachments.

Request body

FIELD	TYPE	REQUIRED	NOTES
`body`	string	Conditional	Max 2000. Required unless `image` is present.
`image`	file	Conditional	JPEG/PNG/WebP, max 8192 KB. Required unless `body` is present. An image-only message with no caption is valid.

Response `201 Created` — the new `MessageResource`. The other party receives a push notification (`swap_message_received`) with preview text `Str::limit(body, 100)`, or the literal `"📷 Photo"` if the message is image-only.

Groups

`GET /group-categories`

Full admin-managed list, not paginated. Shape identical to `GET /listing-categories`.

`GET /groups`

Active (non-deactivated/non-banned) groups. **Paginated**, 20/page, newest first.

Query parameters

PARAM	TYPE	REQUIRED	NOTES
`category`	string	No	Exact match on `groups.category`.
`search`	string	No	`LIKE` match on group name.

Response `200 OK` — paginated `GroupResource` collection.

```
{
  "data": [
    {
      "id": 12,
      "name": "Dhaka Rooftop Growers",
      "slug": "dhaka-rooftop-growers",
      "description": "A community for balcony and rooftop gardeners in Dhaka.",
      "category": "Urban Farming",
      "visibility": "public",
      "verified": true,
      "cover_image_url": "https://codecell.site/storage/groups/....jpg",
      "avatar_url": "https://codecell.site/storage/groups/....jpg",
      "members_count": 214,
      "posts_count": 89,
      "posts_today_count": 3,
      "creator": { "...": "UserResource" },
      "is_member": true,
      "join_request_pending": false,
      "pending_join_requests_count": 0,
      "pinned_message": "Meetup this Saturday at Ramna Park, 9am!",
      "pinned_by": { "...": "UserResource" },
      "pinned_at": "2026-09-15T06:00:00+00:00",
      "unread_count": 4,
      "last_message": {
        "body": "See everyone Saturday!",
        "has_image": false,
        "sender_id": 88,
        "sender_name": "Karim Hossain",
        "created_at": "2026-09-18T14:02:00+00:00"
      },
      "created_at": "2025-11-02T09:00:00+00:00",
      "updated_at": "2026-09-18T14:02:00+00:00"
    }
  ]
}
```


`my_role` is absent here (only present on endpoints that eager-load the full `members` relation — `show` / `store` / `join` / `leave`, not `index`).

`POST /groups`

Creates a group. Caller becomes its `admin` member automatically. `multipart/form-data` for image uploads.

Request body

FIELD	TYPE	REQUIRED	NOTES
`name`	string	Yes	Max 255.
`slug`	string	Yes	Max 255, `alpha_dash`, must be unique.
`description`	string | null	No	Max 2000.
`category`	string | null	No	Must match a `group_categories.name` row.
`visibility`	string	No	`public` (default) or `private`.
`cover_image`	file	No	JPEG/PNG/WebP, max 8192 KB.
`avatar`	file	No	JPEG/PNG/WebP, max 8192 KB.

Response `201 Created` — a `GroupResource`, with `my_role: "admin"` and `is_member: true`.

`GET /groups/{group}`

Response `200 OK` — a `GroupResource` with the full member roster eager-loaded (so `my_role` is populated).

Errors: `403` — `{"message": "This group is no longer available."}` if the group has been deactivated or banned.

`PATCH /groups/{group}`

Only the group's creator or an `admin`-role member (or a site admin) may update it.

Request body — same as create, all `sometimes`, plus:

FIELD	TYPE	REQUIRED	NOTES
`pinned_message`	string | null	No	Omit the key to leave the current pin untouched. Send `null` or `""` to unpin. Any other string pins it and stamps `pinned_by` / `pinned_at` with the caller.

Side effect: switching `visibility` from `private` to `public` immediately converts every pending join request into a real membership (rather than leaving requesters stuck waiting on a barrier that no longer exists).

Response `200 OK` — updated `GroupResource`.

Errors: `403` if the caller doesn't manage the group.

`DELETE /groups/{group}`

Hard delete — cascades to the group's posts, memberships, and events at the database level.

Response `200 OK`

```
{ "message": "Group deleted." }
```

`POST /groups/{group}/join`

Joins a public group instantly, or files a join request against a private one.

Response `200 OK` — a `GroupResource` (public group, instant join). `202 Accepted` for a private group:

```
{ "message": "Your request to join has been sent to the group admin." }
```

Errors

`422` — already a member:

```
{ "errors": { "group": ["You are already a member of this group."] } }
```

`422` — request already pending:

```
{ "errors": { "group": ["Your request to join is already pending approval."] } }
```

`403` — `{"message": "This group is no longer available."}` if deactivated/banned.

`DELETE /groups/{group}/join-requests/me`

Withdraws the caller's own still-pending join request.

Response `200 OK`

```
{ "message": "Join request canceled." }
```

Errors: `404` if there is no pending request to cancel.

`GET /groups/{group}/join-requests`

The group's pending join-request queue, oldest first — admin/creator only. **Paginated**, 20/page.

Response `200 OK`

```
{
  "data": [
    { "id": 41, "user": { "...": "UserResource" }, "requested_at": "2026-09-17T11:00:00+00:00" }
  ]
}
```

Errors: `403` if the caller doesn't manage the group.

`PATCH /groups/{group}/join-requests/{joinRequest}`

Approves or rejects a pending request.

Request body

FIELD	TYPE	REQUIRED
`decision`	string	Yes — `approve` or `reject`.

Response `200 OK` — approving deletes the request row and creates a real membership (no "approved" status is ever stored):

```
{ "message": "Join request approved." }
```

or

```
{ "message": "Join request rejected." }
```

Errors: `403` if the caller doesn't manage the group; `404` if the join request doesn't belong to this group.

`POST /groups/{group}/leave`

Response `200 OK`

```
{ "message": "Left group." }
```

Errors: `422` — the creator cannot leave their own group:

```
{ "errors": { "group": ["The group creator cannot leave their own group."] } }
```

`GET /groups/{group}/members`

The real member roster, admins first, then by join date. **Paginated**, 20/page.

Response `200 OK`


```
{
  "data": [
    {
      "id": 154,
      "name": "Amina Rahman",
      "username": "amina_grows",
      "avatar_url": "https://codecell.site/storage/avatars/....jpg",
      "is_verified": true,
      "experience_level": "intermediate",
      "growing_setup": "rooftop",
      "role": "admin",
      "joined_at": "2025-11-02T09:00:00+00:00"
    }
  ]
}
```

`GET /groups/{group}/messages`

The group's real-time chat. **Paginated**, 50/page, most-recent-first fetched then reversed to chronological order for display. Opening the endpoint advances the caller's own per-member read watermark to the highest message id returned.

Response `200 OK`

```
{
  "data": [
    {
      "id": 900,
      "group_id": 12,
      "body": "See everyone Saturday!",
      "image_url": null,
      "sender": { "...": "UserResource" },
      "sender_is_admin": false,
      "created_at": "2026-09-18T14:02:00+00:00"
    }
  ]
}
```


Errors: `403` if the caller isn't a member of the group.

`POST /groups/{group}/messages`

Request body — identical shape to `POST /swap-offers/{swapOffer}/messages` (`body` / `image`, at least one required).

Response `201 Created` — the new `GroupMessageResource`. Every other member is notified (`group_message_received`, batched into one multicast push).

Errors: `403` if the caller isn't a member.

`GET /groups/{group}/posts`

The group's discussion-thread posts (separate from the general Home Feed — see below). **Paginated**, 20/page, newest first.

A non-admin member sees only `approved` posts plus their own pending/rejected ones; the group's admin sees every post regardless of moderation state.

Response `200 OK` — paginated `PostResource` collection (shape documented under Community Feed below).

Errors: `403` — `{"message": "This group is no longer available."}` if deactivated/banned.

Community Feed

`GET /feed`

The general Home Feed — posts with no `group_id`, newest first. **Paginated**, 20/page.

Query parameters

PARAM	TYPE	REQUIRED	NOTES
`search`	string	No	Matches against post body or author name.
`author_id`	integer	No	Narrows to one author's posts — powers "My Posts" on a profile.
`category`	string	No	Must match a `post_categories.name` row. Omitted = every category.

Response `200 OK` — paginated `PostResource` collection.


```
{
  "data": [
    {
      "id": 3021,
      "group_id": null,
      "body": "My first tomato harvest of the season! 🍅",
      "category": "Harvests",
      "approval_status": "approved",
      "image_url": "https://codecell.site/storage/posts/....jpg",
      "comments_count": 6,
      "likes_count": 24,
      "liked_by_me": true,
      "author": { "...": "UserResource" },
      "group": null,
      "created_at": "2026-09-18T07:00:00+00:00",
      "updated_at": "2026-09-18T07:00:00+00:00"
    }
  ]
}
```


`comments` (an embedded `CommentResource[]`) is only populated on `GET /posts/{post}` (see below), where the `comments` relation is eager-loaded — it's absent elsewhere.

`GET /post-categories`

Full admin-managed list, not paginated. Backs the Home tab's Recent/Disease Alerts/Harvests filter chips.

`POST /posts`

Creates a post — into the general feed if `group_id` is omitted, or into a group's discussion thread if set. `multipart/form-data` for image uploads.

Request body

FIELD	TYPE	REQUIRED	NOTES
`group_id`	integer | null	No	Must reference an existing group. Poster must already be a member.
`body`	string	Yes	Max 5000.
`category`	string | null	No	Must match a `post_categories.name` row. Defaults to `"General"`.
`image`	file	No	JPEG/PNG/WebP, max 8192 KB.

Moderation note: a non-admin member's post into a **private** group starts `approval_status: pending` (invisible to the rest of the group until a group admin approves it). Every other case — Home Feed post, public-group post, or the group's own admin posting — is approved immediately.

Response `201 Created` — a `PostResource`, `liked_by_me: false`.

Errors: `422` if `group_id` is set but the caller isn't a member:

```
{ "errors": { "group_id": ["You must be a member of this group to post in it."] } }
```

`GET /posts/{post}`

Response `200 OK` — a `PostResource` with `comments` (top-level + reply comments, flattened) eager-loaded.

Errors: `403` — `{"message": "This action is unauthorized."}` if the post is site-moderated away (`moderation_status != visible`), or is a pending/rejected group post the caller can't see (not its author, not that group's admin).

`PATCH /posts/{post}`

Only the post's own author may edit it — `group_id`, `author`, and moderation/approval state are never editable here.

Request body

FIELD	TYPE	REQUIRED	NOTES
`body`	string	No	Max 5000.
`category`	string | null	No	Must match `post_categories.name`; `null` resets to `"General"`.
`image`	file	No	New image replaces (and deletes) the old one.
`remove_image`	boolean	No	Clears the image without uploading a new one.

Response `200 OK` — updated `PostResource`.

Errors: `403` if the caller isn't the post's author.

`DELETE /posts/{post}`

Response `200 OK`

```
{ "message": "Post deleted." }
```

Errors: `403` if the caller isn't the post's author (no admin-of-group override for deletion — only the author can delete, staff aside).

`POST /posts/{post}/like` · `DELETE /posts/{post}/like`

Idempotent. Liking sends the author a `post_liked` notification, but only on a genuinely new like — re-liking an already-liked post never re-notifies.

Response `200 OK` — updated `PostResource` with `liked_by_me` reflecting the new state.

`POST /posts/{post}/approve` · `POST /posts/{post}/reject`

The post's group admin/creator moderates a pending (or previously rejected) post.

Response `200 OK`

```
{ "message": "Post approved." }
```

or

```
{ "message": "Post rejected." }
```

Errors: `403` if the caller doesn't manage the post's group.

`GET /posts/{post}/comments`

Flattened one-level comment thread (a reply-to-a-reply is stored under the same top-level `parent_id`, not nested further), oldest first. **Paginated**, 30/page.

Response `200 OK`

```
{
  "data": [
    {
      "id": 501,
      "post_id": 3021,
      "parent_id": null,
      "body": "Beautiful! What variety?",
      "author": { "...": "UserResource" },
      "replying_to_name": null,
      "created_at": "2026-09-18T07:10:00+00:00",
      "updated_at": "2026-09-18T07:10:00+00:00"
    },
    {
      "id": 505,
      "post_id": 3021,
      "parent_id": 501,
      "body": "Sweet Million, from a friend's cutting.",
      "author": { "...": "UserResource" },
      "replying_to_name": "Karim Hossain",
      "created_at": "2026-09-18T07:15:00+00:00",
      "updated_at": "2026-09-18T07:15:00+00:00"
    }
  ]
}
```

Errors: `403` if the post itself isn't visible to the caller (same rule as `GET /posts/{post}`).

`POST /posts/{post}/comments`

Request body

FIELD	TYPE	REQUIRED	NOTES
`body`	string	Yes	Max 2000.
`reply_to_comment_id`	integer | null	No	The specific comment being replied to (top-level or a reply). Server flattens it to the thread's top-level `parent_id`.

Response `201 Created` — the new `CommentResource`.

Notification side effects: the post's author is notified (`post_commented`); if this is a reply, the replied-to comment's own author is **additionally** notified (`comment_replied`) — unless that would duplicate the post-author notification just sent, or notify the replier about their own comment.

Errors: `422` — replied-to comment not found on this post:

```
{ "errors": { "reply_to_comment_id": ["That comment could not be found on this post."] } }
```

`403` if the post isn't visible to the caller.

`DELETE /comments/{comment}`

Either the comment's own author, or the author of the post it's on, may delete it.

Response `200 OK`

```
{ "message": "Comment deleted." }
```

Errors: `403` if the caller is neither.

Events

`GET /event-categories`

Full admin-managed list, not paginated.

`GET /events`

Published, not-yet-expired events, soonest first. **Paginated**, 20/page. "Expired" = the event's `ends_at` (or `starts_at` , if no end time) has already passed — an event already underway keeps showing until it actually ends.

Query parameters

PARAM	TYPE	REQUIRED	NOTES
`month`	string	No	Format `YYYY-MM`. Switches to calendar mode: returns **every** event in that month regardless of expiry (past or future).
`category`	string	No	Must match `event_categories.name`.
`group_id`	integer	No	Narrows to one group's events.

Response `200 OK` — paginated `EventResource` collection.


```
{
  "data": [
    {
      "id": 77,
      "group_id": 12,
      "title": "Composting 101 Workshop",
      "description": "Hands-on composting basics for beginners.",
      "format": "in_person",
      "format_label": "In Person",
      "category": "Workshop",
      "cover_image_url": "https://codecell.site/storage/events/....jpg",
      "location": "Ramna Park, Dhaka",
      "starts_at": "2026-09-27T03:00:00+00:00",
      "ends_at": "2026-09-27T05:00:00+00:00",
      "capacity": 30,
      "status": "published",
      "status_label": "Published",
      "attendees_count": 18,
      "is_saved": false,
      "host": { "...": "UserResource" },
      "group": { "...": "GroupResource" },
      "my_rsvp_status": "going",
      "created_at": "2026-09-10T09:00:00+00:00",
      "updated_at": "2026-09-15T09:00:00+00:00"
    }
  ]
}
```


`attendees_preview` (up to 8 `UserResource` objects, newest RSVP first) is present **only** on `GET /events/{event}`, where the full `attendees` relation is eager-loaded — it's absent on `index` / `saved`.

`GET /events/saved`

The caller's bookmarked events, most-recently-saved first. **Paginated**, 20/page. Unlike `index`, this intentionally does **not** filter out expired events — a saved past event is still something the user chose to keep.

Response `200 OK` — paginated `EventResource` collection, every item `is_saved: true`.

`POST /events`

Request body

FIELD	TYPE	REQUIRED	NOTES
`group_id`	integer | null	No	Must reference an existing group.
`title`	string	Yes	Max 255.
`description`	string | null	No	Max 5000.
`format`	string	Yes	`in_person` | `online` | `hybrid`.
`category`	string | null	No	Must match `event_categories.name`.
`cover_image`	file	No	JPEG/PNG/WebP, max 8192 KB.
`location`	string | null	No	Max 255.
`starts_at`	datetime	Yes	Any parseable date.
`ends_at`	datetime | null	No	Must be after `starts_at`.
`capacity`	integer | null	No	Min 1.
`status`	string	No	`draft` (default) | `published` | `canceled`.

Response `201 Created` — an `EventResource`.

`GET /events/{event}`

Response `200 OK` — an `EventResource` with `attendees_preview` and (when authenticated) `is_saved` / `my_rsvp_status` populated.

`PATCH /events/{event}`

Only the event's host may update it.

Request body — same fields as create, all `sometimes`.

Response `200 OK` — updated `EventResource`.

Errors: `403` if the caller isn't the host.

`DELETE /events/{event}`

Response `200 OK`

```
{ "message": "Event deleted." }
```

Errors: `403` if the caller isn't the host.

`POST /events/{event}/rsvp`

Upserts the caller's attendance status (idempotent — re-RSVPing overwrites the existing pivot row rather than erroring).

Request body

FIELD	TYPE	REQUIRED
`status`	string	Yes — `going` or `interested`.

Response `200 OK` — updated `EventResource` with `my_rsvp_status` set.

`DELETE /events/{event}/rsvp`

Response `200 OK`

```
{ "message": "RSVP canceled." }
```

`POST /events/{event}/save` · `DELETE /events/{event}/save`

Idempotent bookmark / un-bookmark.

Response `200 OK` — updated `EventResource` with `is_saved` reflecting the new state.

Notifications

The in-app notification inbox, backing the bell icon shown on the Home, Events, and Groups tabs. Every mutating action elsewhere in this API (like, comment, reply, swap message, group message) writes a row here and pushes via FCM.

`GET /notifications`

Paginated, 20/page, newest first.

Response `200 OK`

```
{
  "data": [
    {
      "id": 9001,
      "type": "post_liked",
      "title": "Karim Hossain liked your post",
      "body": "My first tomato harvest of the season! 🍅",
      "data": { "post_id": 3021, "group_id": null },
      "actor": { "...": "UserResource" },
      "read_at": null,
      "created_at": "2026-09-19T07:45:00+00:00"
    }
  ],
  "links": { "...": "..." },
  "meta": { "...": "..." }
}
```

`type` is one of: `post_liked`, `post_commented`, `comment_replied`, `swap_message_received`, `group_message_received`.

`GET /notifications/unread-count`

Response `200 OK`

```
{ "unread_count": 5 }
```

`POST /notifications/read-all`

Marks every one of the caller's unread notifications as read.

Response `200 OK`

```
{ "message": "All notifications marked as read." }
```

`POST /notifications/{notification}/read`

Idempotent — marking an already-read notification read again is a no-op (the timestamp doesn't change).

Response `200 OK` — the updated `AppNotificationResource`.

Errors: `403` if the notification doesn't belong to the caller; `404` if the id doesn't exist.

AI Assistant (Agri-AI)

Every endpoint in this section requires an active Agri-AI subscription (trialing counts) — including reading past conversation history. A non-subscriber hitting any `/ai/*` route gets:

```
{ "message": "An active Agri-AI subscription is required." }
```

Status `403`, checked before any other authorization.

`GET /ai/conversations`

The caller's own AI conversations, newest first. **Paginated**, 20/page. `messages` is `[]` on this list endpoint (not eager-loaded here).

Response `200 OK`

```
{
  "data": [
    { "id": 44, "title": null, "messages": [], "created_at": "2026-09-18T06:00:00+00:00", "updated_at"
  ]
}
```

`GET /ai/conversations/{conversation}`

Owner-only — **no admin bypass** on this policy, unlike every other resource in this API. Messages are loaded oldest-first.

Response `200 OK`


```
{
  "data": {
    "id": 44,
    "title": null,
    "messages": [
      { "id": 210, "conversation_id": 44, "role": "user", "body": "My tomato leaves have yellow spots,
      { "id": 211, "conversation_id": 44, "role": "assistant", "body": "That pattern is consistent wit
    ],
    "created_at": "2026-09-18T06:00:00+00:00",
    "updated_at": "2026-09-18T06:00:04+00:00"
  }
}
```


Errors: `403` (subscription gate) or `403` `{"message": "This action is unauthorized."}` (not the owner) — the subscription check runs first, so a non-subscriber always sees the subscription message even for someone else's conversation id. `404` if the id doesn't exist.

`POST /ai/messages`

Sends a message — starts a new conversation if `conversation_id` is omitted, or continues an existing one. `multipart/form-data` for photo-based questions (e.g. leaf-disease diagnosis).

Request body

FIELD	TYPE	REQUIRED	NOTES
`conversation_id`	integer | null	No	Must belong to the caller. Omit to start a new conversation.
`body`	string	Conditional	Max 4000. Required unless `photo` is present.
`photo`	file	Conditional	JPEG/PNG/WebP, max 8192 KB. Required unless `body` is present — a bare photo upload with no caption is a valid "diagnose this" request.

Response `201 Created` — the full `AiConversationResource`, including the newly appended user message and the assistant's synchronous reply.

Errors

`422` — neither field present:

```
{
  "message": "The given data was invalid.",
  "errors": {
    "body": ["The body field is required when photo is not present."],
    "photo": ["The photo field is required when body is not present."]
  }
}
```

`403` — subscription gate, same shape as above.

`500` — the AI backend (Gemini) call fails or returns a non-2xx response; there is no custom "AI service unavailable" error surfaced here, it presents as a generic server error.

Subscriptions & Billing

Agri-AI Premium is powered by Stripe via Laravel Cashier. Payment method tokenization happens client-side (Stripe's mobile SDK) — raw card data never passes through this server.

`GET /subscriptions/me`

Not subscription-gated — this is how the client determines whether the AI features are locked.

Response `200 OK`

```
{
  "data": {
    "has_access": true,
    "plan": "monthly",
    "status": "trialing",
    "on_trial": true,
    "trial_ends_at": "2026-09-26T08:00:00+00:00",
    "cancel_at_period_end": false,
    "renews_at": "2026-09-26T08:00:00+00:00",
    "ends_at": null
  }
}
```

`plan` is `"monthly"` | `"annual"` | `null` (no subscription, or the stored Stripe price no longer matches either configured plan).

`POST /subscriptions`

Starts a new subscription with a 7-day free trial.

Request body

FIELD	TYPE	REQUIRED	NOTES
`plan`	string	Yes	`monthly` or `annual`.
`payment_method`	string	Yes	A Stripe PaymentMethod id (`pm_...`) already tokenized on-device.

Response `201 Created` — a `SubscriptionResource` (same shape as `GET /subscriptions/me`).

Response `202 Accepted` — 3-D Secure confirmation required; the mobile app must complete it client-side using the given secret, then the subscription activates automatically via Stripe webhook (no re-call of this endpoint needed):

```
{
  "requires_action": true,
  "payment_intent_client_secret": "pi_3P...secret_xyz"
}
```

Errors

`503` — no Stripe price configured yet for the requested plan (admin hasn't set one up):

```
{ "message": "Subscription plans are not configured yet. Please try again shortly." }
```

`422` — invalid `plan` value or missing `payment_method`.

`500` — any other Stripe API error (card declined, invalid payment method, etc.) is not specially handled and surfaces as a generic server error.

`DELETE /subscriptions/me`

Cancels **at the end of the current paid (or trial) period** — the caller keeps access until then; this is not an immediate revocation. Idempotent (no error if there's no subscription, or it's already canceled).

Response `200 OK` — the refreshed `SubscriptionResource`, now with `cancel_at_period_end: true` and `ends_at` populated.